

Lecture 04

Introduction to Convolutional Neural Networks

From Mathematical Convolution to NumPy, PyTorch and TensorFlow

Lecturer: Dinh Que Tran

Intelligent System Development

Lecture 04: Where Are We?

In the previous lecture we learned:

Deep Learning = Composition of learned functions

For example:

$$X \rightarrow H_1 \rightarrow H_2 \rightarrow \hat{y}.$$

We implemented a dense neural network from scratch.

Today we ask:

Can we design a layer that discovers local patterns?

Convolution

From Dense Networks to CNN

A dense layer connects every input feature to every neuron.

$$H = \sigma(XW + b).$$

This is powerful, but it ignores local structure.

For images:

pixel at (i, j)

is naturally related to nearby pixels:

$$(i-1, j), (i+1, j), (i, j-1), (i, j+1).$$

For text:

word _{i}

is related to nearby words.

The Key Idea of Convolution

Suppose we have a 1D signal:

$$x_1, x_2, x_3, x_4, x_5, x_6.$$

Consider a kernel:

$$w_1, w_2, w_3.$$

The kernel slides across the input.

$$z_i = w_1x_i + w_2x_{i+1} + w_3x_{i+2}$$

for each valid position.

Thus:

$$[x_1, x_2, x_3] \rightarrow z_1$$

$$[x_2, x_3, x_4] \rightarrow z_2$$

Convolution as a Function

A convolution layer is another parameterized function.

For one-dimensional input:

$$Z = \text{Conv1D}(X; W, b)$$

Then normally:

$$H = \text{ReLU}(Z).$$

Therefore:

$$f(X) = \text{ReLU}(\text{Conv1D}(X; W, b))$$

This fits directly into our previous idea:

$$\hat{y} = f_3(f_2(f_1(X)))$$

Why Does Convolution Help?

A convolution has two important properties.

① **Local connectivity**

The kernel sees only a small neighborhood.

② **Weight sharing**

The same kernel is reused at different positions.

Therefore:

$$\text{CNN} = \text{Locality} + \text{Weight Sharing} + \text{Nonlinearity}$$

This makes CNNs particularly useful for:

- images
- audio
- time series
- sequences
- text

A Simple 1D CNN

Consider:

$$X \in \mathbb{R}^8.$$

This is convenient for our diabetes dataset.

We can regard the eight features as a short 1D sequence:

$$[x_1, x_2, \dots, x_8].$$

Then:

8 $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Dense $\rightarrow$ Prediction

Important Modeling Warning

Diabetes and house-price datasets are **tabular**.

Their columns do not necessarily possess the spatial locality of pixels.

For example:

Glucose

being next to

BloodPressure

in the CSV file does not mean that these variables have a spatial relationship.

Therefore, using a 1D CNN on tabular data is:

a teaching experiment

rather than a claim that CNN is the optimal model.

Comments/text are much more natural for 1D convolution because nearby tokens can form

Three Datasets

Exercise 04 will use three different types of data.

Dataset	Data Type	CNN Interpretation
Diabetes	Tabular	1D feature sequence
House Price	Tabular	1D feature sequence
Comments	Text	1D token sequence

The goal is not merely accuracy.

The goal is to understand:

How the same Deep Learning idea adapts to different data

CNN Architecture for Diabetes

A simple teaching architecture:

$$8 \rightarrow 16 \rightarrow 16 \rightarrow 8 \rightarrow 1.$$

More explicitly:

Input $\rightarrow$ Conv1D $\rightarrow$ ReLU $\rightarrow$ Conv1D $\rightarrow$ ReLU $\rightarrow$ Pooling $\rightarrow$ Dense $\rightarrow$ Sigmoid

The first convolution learns local combinations of patient features.

The second convolution learns combinations of those local patterns.

CNN Architecture for Comments

For text:

tokens $\rightarrow$ embedding $\rightarrow$ Conv1D $\rightarrow$ ReLU $\rightarrow$ Pooling $\rightarrow$ Dense $\rightarrow$ classification.

For example:

[this, product, is, very, good]

may contain local patterns such as:

[very, good]

or:

[not, good].

The convolution can learn such local patterns.

Convolution Kernel

Suppose:

$$X = [2, 1, 3, 4, 2].$$

A kernel is:

$$W = [0.5, -1, 0.5].$$

At the first position:

$$z_1 = 2(0.5) + 1(-1) + 3(0.5).$$

Therefore:

$$z_1 = 1.$$

The kernel then moves:

$$[2 \ 1 \ 3] \rightarrow [1 \ 3 \ 4] \rightarrow [3 \ 4 \ 2]$$

Convolution + ReLU

After convolution:

$$Z = \text{Conv}(X, W).$$

Then:

$$H = \text{ReLU}(Z).$$

Therefore:

$$H = \text{ReLU}(\text{Conv}(X, W) + b)$$

This is directly analogous to:

$$H = \text{ReLU}(XW + b)$$

in a dense layer.

The major difference is the structured local operation.

Pooling

A common operation is max pooling.

For example:

[1, 5, 2, 4]

with pool size 2 gives:

[5, 4].

Mathematically:

$$p_i = \max(x_i, x_{i+1})$$

Pooling reduces the representation size.

It can also make the representation less sensitive to small positional changes.

A 3-Layer CNN

For Exercise 04, students will implement a small network.

Conv1 $\rightarrow$ ReLU $\rightarrow$ Conv2 $\rightarrow$ ReLU $\rightarrow$ Dense

This is already a composition:

$$f(X) = f_3(f_2(f_1(X))).$$

The trainable functions are:

$$f_1 = \text{Conv1D}$$

$$f_2 = \text{Conv1D}$$

$$f_3 = \text{Dense}.$$

A 5-Layer CNN

Students may extend the architecture to:

Conv1 → ReLU → Conv2 → ReLU → Pool → Conv3 → ReLU → Dense → Output

The exact definition of “layer” should be stated explicitly in the report.

For this course:

trainable convolution/dense transformations count as layers

CNN From Scratch

Before using PyTorch or TensorFlow, students implement the **convolution operation manually**.

Mathematical Definition

For input x and kernel k of size m :

$$z_i = \sum_{j=0}^{m-1} k_j x_{i+j} + b$$

The kernel moves across the input and produces one value at each position.

CNN From Scratch: Numerical Example

Consider

$$x = [2, 1, 3, 4, 2]$$

and kernel

$$k = [0.5, -1, 0.5], \quad b = 0.$$

The kernel size is 3.

$$\boxed{[2, 1, 3]} \quad [4, 2]$$

$$z_1 = (2)(0.5) + (1)(-1) + (3)(0.5)$$

From Convolution to a CNN Layer

The convolution operation alone is not yet a complete neural network layer.

$$z = \text{Conv1D}(x; k, b)$$

Apply a nonlinear activation:

$$h = \text{ReLU}(z)$$

where

$$\text{ReLU}(z) = \max(0, z).$$

For the previous example:

$$z = [1, -0.5, -1.5]$$

therefore

Why Implement CNN From Scratch?

The Main Reason

A framework can make CNN look like:

```
model = Conv1D(...)
```

But students should understand what happens **inside** that function.

- What is a convolution?
- How does a kernel slide over the input?
- Where do the parameters come from?
- How is the loss computed?
- How are gradients obtained?

From Mathematics to Intelligence

$$X \xrightarrow{\text{Convolution}} Z \xrightarrow{\text{ReLU}} H \xrightarrow{\text{Dense}} \hat{y}$$

Training:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

where

$$\theta = \{K, b, W, c\}.$$

From Scratch to PyTorch

The manually implemented operation

$$\text{Conv1D}(X, W, b)$$

becomes conceptually:

```
nn.Conv1d(  
    in_channels,  
    out_channels,  
    kernel_size  
)
```

The activation:

```
nn.ReLU()
```

and pooling:

```
nn.MaxPool1d(...)
```

From Scratch to TensorFlow

The same conceptual operation can be expressed using TensorFlow/Keras:

```
tf.keras.layers.Conv1D(  
    filters=16,  
    kernel_size=3,  
    activation="relu"  
)
```

Then pooling:

```
tf.keras.layers.MaxPooling1D(...)
```

and the output layer for binary classification:

```
tf.keras.layers.Dense(  
    1,  
    activation="sigmoid"  
)
```

The Three Implementations

MATHEMATICS



NUMPY FROM SCRATCH



PYTORCH

What Stays the Same?

Across all three implementations:

- Input data
- Train/test split
- Preprocessing
- Model idea
- Loss function
- Evaluation metrics
- Basic optimization objective

Therefore:

Fair comparison requires controlled experiments

What Changes?

The implementation layer changes.

NumPy	PyTorch	TensorFlow
Manual convolution	Conv1d	Conv1D
Manual ReLU	ReLU	ReLU
Manual gradients	Autograd	GradientTape
Manual update	Optimizer	Optimizer
Manual training loop	Training loop	fit() / custom loop

This is an important software-engineering lesson:

Same mathematical model $\neq$ same implementation

Evaluation

Students will evaluate:

Accuracy

Precision

Recall

F1

and training behavior:

Loss vs. Epoch

For regression:

MSE, RMSE, R2

Classification vs. Regression

The three datasets may have different tasks.

Dataset	Task	Output
Diabetes	Classification	$P(y = 1)$
Comments	Classification	class probability
House Price	Regression	predicted price

Therefore, house-price prediction should use:

Linear output neuron

rather than sigmoid.

The loss may be:

MSE

or:

A Very Important Lesson

The architecture should follow the task.

For binary classification:

Output: Sigmoid

For multiclass classification:

Output: Softmax

For regression:

Output: Linear

Thus:

Neural network architecture = Data + Task + Representation

Lecture 04 Summary

Today we moved from:

Dense Layer

to:

Convolution

and then:

CNN.

The key concepts are:

- 1 Local connectivity
- 2 Weight sharing
- 3 Learned filters
- 4 ReLU
- 5 Pooling

The Central Equation

A CNN is still a composition of functions:

$$\hat{y} = f_L \circ f_{L-1} \circ \dots \circ f_2 \circ f_1(X)$$

The difference is that some of these functions are convolutional.
Therefore:

CNN is not a completely different idea from Deep Learning

It is a particular architecture for learning useful representations.

Lecture 03

Function Composition



Lecture 04

Convolutional Function Composition

Build, Run, Evaluate and Compare CNNs

Students will implement CNN models in:

NumPy from scratch

PyTorch

TensorFlow

using:

Diabetes + House Price + Comments

Do not learn PyTorch first.

Understand the function first.

Mathematics $\rightarrow$ NumPy $\rightarrow$ PyTorch $\rightarrow$ TensorFlow

This sequence makes Deep Learning understandable rather than magical.

“‘latex

CNN Structure: Three Implementations

$X \rightarrow \text{Conv1D} \rightarrow \text{ReLU} \rightarrow \text{Conv1D} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \text{Flatten} \rightarrow \text{Dense} \rightarrow \hat{y}$

Operation	NumPy	PyTorch	TensorFlow/Keras
Convolution	<code>conv1d()</code>	<code>nn.Conv1d()</code>	<code>layers.Conv1D()</code>
Activation	<code>relu()</code>	<code>nn.ReLU()</code>	<code>activation="relu"</code>
Pooling	<code>maxpool1d()</code>	<code>nn.MaxPool1d()</code>	<code>MaxPooling1D()</code>
Flatten	<code>x.reshape()</code>	<code>torch.flatten()</code>	<code>Flatten()</code>
Dense	<code>dense()</code>	<code>nn.Linear()</code>	<code>Dense()</code>
Gradient	Manual	Autograd	Automatic

CNN Structure: What Changes?

NumPy

We implement

$$z = \text{Conv}(x)$$

$$h = \text{ReLU}(z)$$

$$p = \text{Pool}(h)$$

$$\hat{y} = f(p)$$

Students see the
mechanism.

PyTorch

Framework provides

$$z = \text{Conv1d}(x)$$

$$h = \text{ReLU}(z)$$

$$p = \text{Pool}(h)$$

Students focus on **model
design.**

TensorFlow

Framework provides

$$z = \text{Conv1D}(x)$$

$$h = \text{ReLU}(z)$$

$$p = \text{Pool}(h)$$

Students focus on **model
design.**

CNN Structure: What Does Not Change?

Regardless of implementation, the learning system contains:



$$\hat{y} = f_{\theta_3}(\text{Pool}[\text{ReLU}(f_{\theta_2}(\text{ReLU}(f_{\theta_1}(X))))])$$

Important

PyTorch and TensorFlow change the **implementation**, not the fundamental learning architecture.